SOFTWARE REQUIREMENTS SPECIFICATION

MINI SPOTIFY(Simple Stream): a simplified Spotify-like music streaming app

Distributed Computing | University of Hyderabad

Student	K. Amulya	**Reg. No.**	23MCCE22
Based on	Spotify (music streaming service)	**Proposed app**	SimpleStream (simplified version, in Java)

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) states what the simplified Spotify-like application, **SimpleStream v1.0**, shall do. It is written for the course evaluators, for developers who will maintain or extend the code, and for testers who need requirements to check against. It follows the supplied IEEE-830-style template [1] and is based on the project I developed, its source code and its README.

1.2 Scope

Product name: SimpleStream. **What it is:** a music streaming application with a website and a terminal client, connected to a cluster of three socket-server nodes. It lets a registered user search an 18-song catalogue, play songs, like songs and manage playlists, and it keeps each user's data on disk. It is the **simplified version** of a service such as Spotify: it keeps the core listener experience and the core distributed-systems ideas (load balancing, replication, failover, logical clock), and leaves out everything that needs commercial infrastructure or licensed content [3][4].

1.3 Definitions, Acronyms, Abbreviations

Term	Meaning
SRS	Software Requirements Specification.
Node	One `MusicServer` instance listening on port 5001, 5002 or 5003.
Balancer	Client-side component (`Balancer.java`) that picks a node by round robin and fails over to the next node.
REPL	Replication message `REPL<TAB>clock<TAB>command` that a node sends to its peers after a write.
Lamport clock	Logical counter used to order events across nodes: `clock = max(own, received) + 1`.
Write	A command that changes data: `SIGNUP`, `LIKE`, `PLCREATE`, `PLDELETE`, `PLADD`, `PLREMOVE`, `PLAY`.
Range request	HTTP request for part of a file (206 Partial Content), used by the browser to seek in audio.
TCP / HTTP / JDK	Transmission Control Protocol / Hypertext Transfer Protocol / Java Development Kit.

1.4 References

[1] Helios (Spotify's earlier open-source container orchestrator) — Spotify GitHub

[2] Apache Cassandra documentation — gossip, consistent hashing, tunable consistency

[3] Apache Kafka documentation — replication and partition leaders

[4] Kubernetes documentation — etcd and the Raft consensus algorithm

[5] Backend infrastructure at Spotify — Spotify Engineering (2013)

[6] Achieving Fault-Tolerance with Intelligent Daemons — Spotify Engineering (2013)

[7] Spotify's Event Delivery – The Road to the Cloud, Part I — Spotify Engineering (2016)

1.5 Overall Description

Section 2 describes the product in context: its place in context, its functions, its users, its constraints and the release plan. Section 3 gives the detailed requirements: interfaces, functions with use cases and data, performance, the database, design constraints and quality attributes. Section 4 gives supporting information: version history, traceability, acceptance tests and open issues.

2. OVERALL DESCRIPTION

2.1 Product perspective

SimpleStream is a self-contained product; it does not use any external music service or content. It has three layers on one computer: clients (website and terminal), a gateway with a client-side balancer, and a cluster of three identical `MusicServer` nodes that keep the data and replicate writes (Figure 1). The nodes store data in two text files; there is no database server.

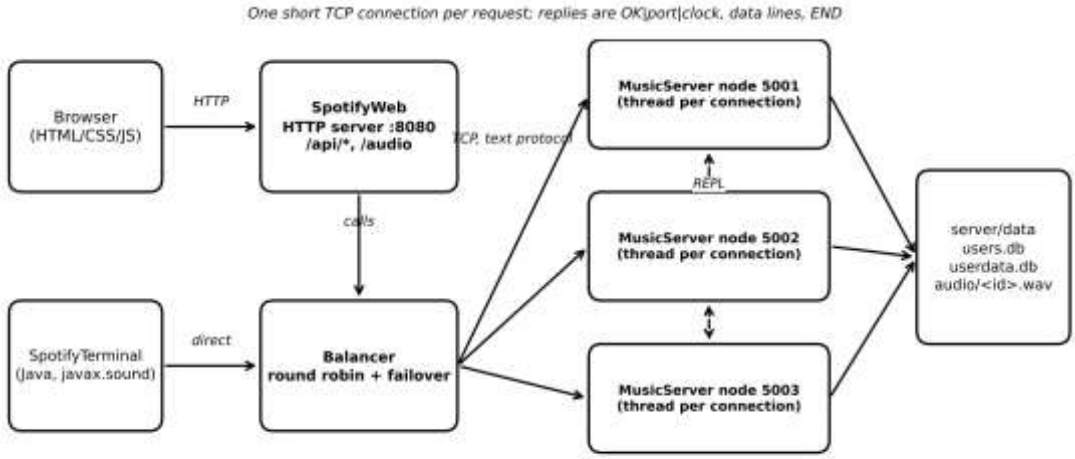


Figure 1. System context and architecture of SimpleStream v1.0.

Layer	Component	Role
Client	Browser (`public/`), `SpotifyTerminal`	Show screens and take user input; play audio.
Gateway	`SpotifyWeb` (port 8080)	Serve the website, turn `/api/*` calls into backend commands, stream audio with Range support.
Balancer	`Balancer` (inside each client)	Round-robin choice of node; failover to the next node.
Application	`MusicServer` nodes 5001, 5002, 5003	Accounts, likes, playlists, history, play counts; replication; Lamport clock.
Data	`users.db`, `userdata.db`, `audio/`	Accounts, user data and song files.

2.2 Product functions

ID	SimpleStream function
F1	Sign up with email, username and password
F2	Log in and log out; session remembered in the browser
F3	Browse all songs grouped by language
F4	Search by title, artist or language
F5	Home: Trending Songs, recently played, most played, picks per language
F6	Play a song with seek, pause, next, previous
F7	Like or unlike songs; Library
F8	Create, open, edit and delete playlists
F9	3-node cluster with load balancing, failover and replication

ID	SimpleStream function
F10	Node status and “Crash last node” demo
F11	Terminal client with text commands
F12	Use own mp3 files and generated WAV tunes
F13	Persist accounts and user data across restarts

2.4 User constraints

- Website login uses **email + password**; accounts made in the terminal have no email and can only log in from the terminal.
- Username must be 3–15 letters, digits or _; password at least 3 characters; email must be valid and unused.
- The terminal client plays **WAV only**; the website plays WAV first and mp3 if no WAV exists.
- Songs are identified by their position in `Catalog.java`; inserting songs in the middle would change saved ids, so new songs are added at the end.
- Free ports 8080, 5001, 5002 and 5003 are needed on the user’s computer.

2.5 Assumptions and dependencies

- JDK 11 or newer is installed; no other software, database, framework or internet connection is needed.
- The user’s browser supports HTML5 audio, `fetch()` and `localStorage`.
- All three nodes run on one computer; clients and nodes can reach each other on localhost.
- A node that crashes is restarted together with the whole backend; it is not re-synchronised alone [16].

3. SPECIFIC REQUIREMENTS

3.1 Interface requirements

3.1.1 External Interface

Website (browser). The page has a login card with Log In and Sign Up tabs, a sidebar, a main area and a player bar.

Screen	Content
Login / Sign Up card	Tabs; email, username, password; messages such as “Account created”
Sidebar	Home, Search, Library, Playlists, All Songs; the 3 nodes (up/down, last served); user name; Log out; “Crash last node”
Home	Recently played (up to 6), most played (top 5, everyone), 3 picks per language
Search	Text box; results update as you type
Library, All Songs, Playlists	Song cards with cover symbol, play button, heart and “+ Playlist”; playlist list with create and delete
Player bar	Play/Pause, Previous, Next, seek bar; visible on every screen

Terminal client. Text commands entered at a prompt; the first screen asks the user to sign up or log in.

Group	Commands
Account	`signup <user> <pass>` · `login <user> <pass>` · `logout` · `whoami`
Browse	`home` · `songs` · `search <text>` · `library` · `like <id>`
Playlists	`playlists` · `open <name>` · `create <name>` · `delete <name>` · `add <songId> <playlist>` · `remove <songId> <playlist>`
Playback	`play <id>` · `pause` · `resume` · `next` · `prev` · `now`
Cluster / other	`nodes` · `crash <port>` · `help` · `quit`

3.1.2 Hardware Interface

- **Computer:** any computer able to run JDK 11 (no special CPU); about 512 MB of free memory is sufficient for three nodes and the web server (design assumption, not measured).
- **Audio output:** speakers or headphones; the terminal client uses `javax.sound`, the website uses the browser's audio.
- **Network:** loopback (localhost) only; no external network is needed.
- **Storage:** a few megabytes for 18 generated WAV files and the two `.db` files.

3.1.3 Software Interface

Interface	**SimpleStream v1.0**
Language / runtime	Java, JDK 11+ (`java.net`, `java.io`, `javax.sound`)
Operating system	Windows (`run.bat`), Linux and Mac (`run.sh`)
Browser	Any modern browser (HTML5 audio, `fetch`, `localStorage`)
Database	None; text files
Security library	`MessageDigest` SHA-256
Third-party libraries	None

3.1.4 Communication Interface

HTTP (browser to SpotifyWeb, port 8080). Static files, `/audio/<id>` and `/api/<name>` routes. The 16 API routes map to backend commands:

API route (`/api/`…)	**Backend command and parameters**
`signup`, `login`	`SIGNUP user pass email`, `LOGIN user pass`
`songs`, `search`, `home`, `library`	`SONGS`, `SEARCH q`, `HOME`, `LIBRARY` (each with the logged-in user)
`like`	`LIKE id` (toggle)
`playlists`, `playlist`, `create`, `delete`	`PLAYLISTS`, `PLSONGS name`, `PLCREATE name`, `PLDELETE name`
`add`, `remove`	`PLADD name id`, `PLREMOVE name id`
`play`, `next`, `prev`	`PLAY id`, `NEXT id playlist`, `PREV id playlist`

Backend command reference. `<u>` is the logged-in username, added by the gateway to every command except `SIGNUP`, `LOGIN` and `PING`.

Command	**Arguments**	**Reply data**	**Write / replicated**
`SIGNUP`	user, pass, email (optional)	`M|Account created…`	Yes
`LOGIN`	email or user, pass	`U|username`, `M|Welcome back…`	No
`SONGS`	u	`#<Language> Songs` then `S|…` lines	No
`SEARCH`	u, text	`#Search results`, `S|…` lines	No
`HOME`	u	Recently Played, Most Played, Picks	No
`LIBRARY`	u	`#Liked Songs`, `S|…` lines	No
`LIKE`	u, id	`M|Added to / Removed from Library`	Yes
`PLAYLISTS`, `PLSONGS`	u; u, name	`P|name|count`; `#name`, `S|…`	No
`PLCREATE`, `PLDELETE`	u, name	`M|Created / Deleted playlist`	Yes
`PLADD`, `PLREMOVE`	u, name, id	`M|Added / Removed …`	Yes
`PLAY`	u, id	`#Now Playing`, `S|…`	Yes

Command	Arguments	Reply data	Write / replicated
`NEXT`, `PREV`	u, id, playlist	`#Next` / `#Previous`, `S|…`	No
`PING`, `CRASH`	none	`OK|port|clock`	No
`REPL`	clock, command…	(node-to-node)	Replica of a write

TCP text protocol (components to nodes, ports 5001–5003). A request is `COMMAND<TAB>arg<TAB>arg` plus a newline. A reply is `OK|port|clock`, then data lines (`S|id|title|artist|language|liked|plays`, `P|name|count`, `M|message`, `#heading`), then `END`; an error is `ERR|message`. One short connection is used per request, with 800 ms connect and 4 s read timeouts. Between nodes, a write is forwarded as `REPL<TAB>clock<TAB>command` (500 ms connect, 1 s read).

3.2 Functional requirements

3.2.1 Use case model / Information Flows

Figure 2 shows the actors and use cases. The Listener uses the website; the Terminal user can do the same through commands; the Operator starts the system and runs the demonstrations.

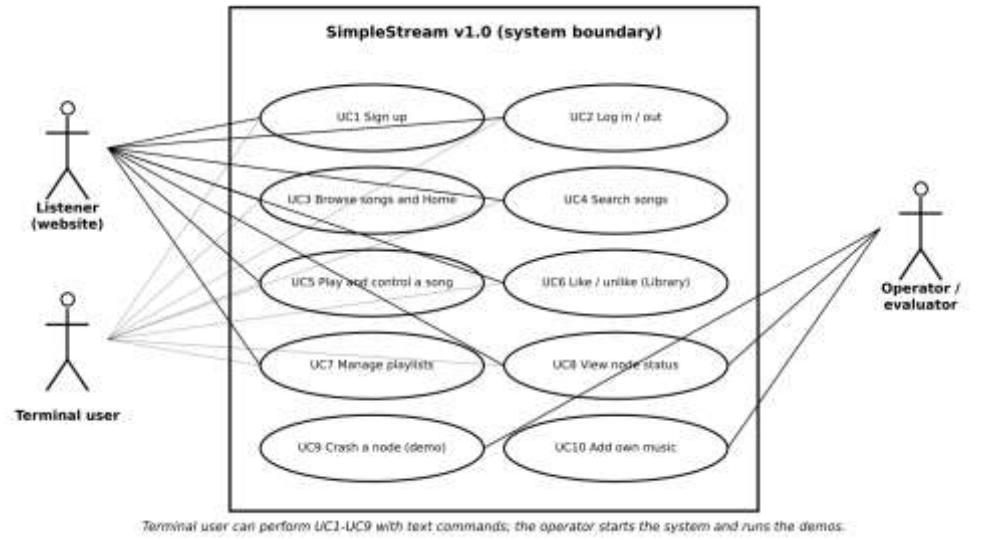


Figure 2. Use case diagram of SimpleStream v1.0.

Figure 3 shows the information flows: every request passes through the gateway and the balancer to one node, which reads or writes `users.db` and `userdata.db`, replicates writes to the other nodes and returns a reply; audio files are read from `audio/`.

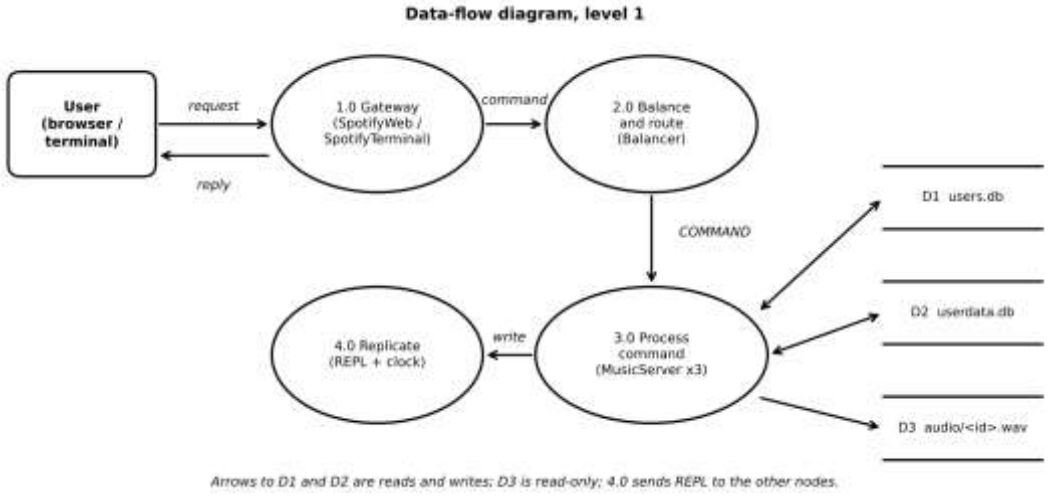


Figure 3. Data-flow diagram (level 1).

3.2.2 Use Case Specifications / Process description

UC1	**Sign up**
Actor	Listener / Terminal user
Precondition	Backend running; user has no account.
Main flow	1. Enter email, username, password. 2. System validates them. 3. System stores the account (SHA-256 hash) and replicates the write. 4. “Account created” is shown and the Log In tab opens.
Alternative / exception	Invalid email, username not 3–15 characters, password under 3 characters, username or email already used → error message, no account created.
Postcondition	Account saved in `users.db` on every node.

UC2	**Log in / Log out**
Actor	Listener / Terminal user
Precondition	Account exists.
Main flow	1. Enter email (or username in the terminal) and password. 2. System compares the hash. 3. System returns the username; the browser keeps it in `localStorage`. 4. Log out clears it.
Alternative / exception	Unknown email or wrong password → “Wrong email or password”.
Postcondition	User is identified on every later request.

UC3–UC4	**Browse songs, Home and Search**
Actor	Listener / Terminal user
Precondition	Logged in.
Main flow	1. Open Home, All Songs or Search. 2. System returns song lines for the user (with liked flag). 3. Search matches the text against title, artist and language, ignoring case.
Alternative / exception	No match → empty result list.
Postcondition	Nothing is changed.

UC5	**Play and control a song**
Actor	Listener / Terminal user
Precondition	Logged in; song exists.
Main flow	1. Click play (or `play <id>`). 2. Node increments the song’s play count and puts it first in Recently Played (max 6); the write is replicated. 3. Browser requests `/audio/<id>`; server answers with the byte range (206) so seeking works. 4. Next/Previous send the current song and playlist; the node answers with the following song, wrapping around.
Alternative / exception	Unknown song id → “No such song”; if the song is not in the chosen playlist the whole catalogue order is used.
Postcondition	Audio plays; history and counts updated on all nodes.

UC6	**Like / unlike**
Actor	Listener / Terminal user
Precondition	Logged in.
Main flow	1. Click the heart (or `like <id>`). 2. Node toggles membership in the user’s liked set and replicates it. 3. Library shows liked songs.
Alternative / exception	Unknown id → error.
Postcondition	Liked set saved in `userdata.db`.

UC7	**Manage playlists**

Actor	Listener / Terminal user
Precondition	Logged in. Every user starts with a playlist “My Favourites”.
Main flow	1. Create a playlist with a name. 2. Add songs with “+ Playlist”. 3. Open a playlist to see its songs. 4. Remove a song or delete the playlist.
Alternative / exception	Empty or duplicate playlist name, song already present, unknown playlist or song → error.
Postcondition	Playlist data saved and replicated.

UC8–UC9	**View node status and crash a node (demo)**
Actor	Operator / evaluator
Precondition	Backend running with 3 nodes.
Main flow	1. Sidebar polls the nodes with `PING` and shows up or down plus the last node that served a request. 2. “Crash last node” (or `crash <port>`) sends `CRASH`; that node closes. 3. The user keeps using the app; `Balancer` fails over to the other nodes.
Alternative / exception	If all nodes are down → “All backend nodes are down”.
Postcondition	Service continues on the remaining nodes; restart the whole backend to recover the crashed node.

UC10	**Add own music**
Actor	Operator / developer
Precondition	JDK installed; project built.
Main flow	1. Name the mp3 by song id and place it in `server/data/audio/`. 2. Delete the matching `.wav`. 3. Edit title and artist in `Catalog.java`; rebuild and restart.
Alternative / exception	Terminal client cannot play mp3.
Postcondition	The website plays the new file.

Process description: system start-up. (1) `run build` compiles `server/src` and `client/src` into `server/out`. (2) `run web` starts `MusicServer`, which first generates the 18 WAV files if they are missing, loads `users.db` and `userdata.db`, and opens the three nodes on ports 5001–5003, each loading the same data. (3) After a short wait, `SpotifyWeb` opens port 8080 and serves `public/`. (4) The browser loads the login page; the sidebar starts polling the nodes. (5) The terminal client can be started at any time with `run terminal`.

Process description: one request. The browser calls `/api/<name>`; `SpotifyWeb` looks the name up in its 16 routes and builds the command; `Balancer.send()` picks a node; the node locks, applies the command and, for a write, saves to disk; the node forwards the write to the other nodes and returns the reply; `SpotifyWeb` converts the reply lines to JSON for the page.

Functional requirements. Each requirement is stated as “The system shall …”.

ID	**Requirement (SimpleStream v1.0)**
FR-01	Register an account with a valid unused email, a unique 3–15 character username (letters, digits, _) and a password of 3 or more characters; store only the SHA-256 hash.
FR-02	Log in with email and password (website) or username and password (terminal); reject wrong credentials.
FR-03	Keep the website user logged in across a page refresh and provide Log out.
FR-04	List all 18 songs grouped by language (Hindi, English, Telugu).
FR-05	Search songs by title, artist or language, case-insensitively.
FR-06	Show Home: last 6 played, 5 most played by all users, 3 picks per language.
FR-07	Play a song, count the play, update Recently Played, and stream audio with HTTP Range support for seeking.
FR-08	Provide Play/Pause, Next, Previous (wrapping, within the playlist when given) and a seek bar.

ID	Requirement (SimpleStream v1.0)
FR-09	Toggle like on a song and list liked songs in Library.
FR-10	Create, list, open and delete playlists; add and remove songs without duplicates.
FR-11	Keep each user's likes, playlists and history separate.
FR-12	Save accounts and user data to disk after every write and reload them at start-up.
FR-13	Run three nodes; distribute requests by round robin; fail over to the next node if one does not answer.
FR-14	Replicate every write to the other nodes with a Lamport clock value; skip nodes that are down.
FR-15	Show node status (up/down, last served) and allow a node to be crashed for demonstration.
FR-16	Provide the terminal client commands listed in 3.1.1 and print the serving node and logical clock.
FR-17	Generate one WAV tune per song at first start and accept replacement mp3 files named by song id.

Validation rules and error messages. The backend answers errors as `ERR|message`; the website shows the message in the page and the terminal prints it.

Situation	Message shown	Command
Email not valid	Please enter a valid email address	`SIGNUP`
Username not 3–15 of letters, digits, _	Username must be 3-15 letters/digits	`SIGNUP`
Password under 3 characters	Password needs at least 3 characters	`SIGNUP`
Username already used	Username already taken	`SIGNUP`
Email already used	Email already registered - please log in	`SIGNUP`
Wrong credentials	Wrong email or password	`LOGIN`
Not logged in	Please log in first	any user command
Song id not 1–18	No such song	`LIKE`, `PLAY`, `PLADD`, `NEXT`, `PREV`
Playlist missing	No such playlist	`PLSONGS`, `PLDELETE`, `PLADD`, `PLREMOVE`
Playlist name empty or used	Playlist name is empty / Playlist already exists	`PLCREATE`
Song already / not in playlist	Song is already in that playlist / Song is not in that playlist	`PLADD`, `PLREMOVE`
All nodes unreachable	All backend nodes are down. Start MusicServer first.	any (from `Balancer`)

3.3 Logical database requirements

There is no database server. Data is held in memory on each node and saved to two text files.

File / record	Content	Written when
`users.db`	One line per account: username, email, SHA-256 hash	Sign up
`userdata.db` record `L`	`L<TAB>user<TAB>ids`: liked songs	Like, any write
`userdata.db` record `R`	`R<TAB>user<TAB>ids`: recently played	Play
`userdata.db` record `P`	`P<TAB>user<TAB>name<TAB>ids`: a playlist	Playlist change
`userdata.db` record `S`	`S<TAB>songId<TAB>plays`: play count	Play

3.5 Design Constraints

Folder / file	Content
server/src/simplestream/	MusicServer, Catalog, Song, AudioGenerator
client/src/simplestream/	SpotifyTerminal, Balancer, , SpotifyWeb,
public/	index.html, css/style.css, js/app.js, img/spotify_logo.png
server/data/	audio/ (WAV or mp3 per song id), users.db, userdata.db
server/out/	Compiled classes created by run build
run.bat, run.sh, README.md	Build and start commands; project description and limitations

3.6 Software System attributes

3.6.1 Reliability

QR-01 Data shall not be lost on a clean restart: every write is saved atomically and reloaded at start-up.

QR-02 A failure of one or two nodes shall not stop the service: Balancer skips dead nodes and replication skips them too.

QR-03 A crash during saving shall not corrupt userdata.db: the new file is complete before it replaces the old one.

3.6.2 Availability

QR-04 While at least one node is up, every request shall be answerable, because any node holds all data.

QR-05 The sidebar shall show which nodes are up.

QR-06 A crashed node shall return only after the whole backend is restarted (known limitation).

3.6.3 Security

ID	Requirement (SimpleStream v1.0)
QR-07	Passwords shall be stored only as SHA-256 hashes, never as plain text (unsalted in v1.0).
QR-08	Inputs shall be validated: username pattern, email pattern, password length, playlist names cleaned of |, tabs and newlines.
QR-09	Each request shall carry the username and each user shall see only their own data (known limitation: no session token).

3.6.4 Maintainability

QR-11 The code shall stay small and readable: seven source files, one class per role, comments at each mechanism.

QR-12 README.md shall describe running, files, protocol and limitations.

QR-13 run.bat and run.sh shall build and start the system with two commands.

QR-14 New songs shall be added at the end of Catalog.SONGS.

3.6.5 Portability

QR-15 The system shall run unchanged on Windows, Linux and macOS with JDK 11 or newer.

QR-16 The website shall work in any modern browser with HTML5 audio.

QR-17 Paths and ports shall be the only environment assumptions.

4. SUPPORTING INFORMATION

4.1 Acceptance test cases

ID	Steps	Expected result
T1	Sign up with a valid email, username alice, password 1234	“Account created”; Log In tab opens

ID	Steps	Expected result
T2	Log in with the email and a wrong password, then the right one	“Wrong email or password”, then welcome and Home
T3	Open All Songs	18 songs in Hindi, English and Telugu groups
T4	Search “arjit”	Songs by that artist only; empty box restores all
T5	Play a song and drag the seek bar	Audio plays and seeks; song appears under Recently Played
T6	Press Next at the last song of a playlist	Wraps to the first song
T7	Click the heart, open Library, click again	Song appears, then disappears
T8	Create a playlist, add a song twice	Second add rejected as already present
T9	Stop and restart the backend; log in	Accounts, likes, playlists and history are still there
T10	Make 6 requests and watch the sidebar	Last served node cycles 5001 → 5002 → 5003
T11	Press “Crash last node”, then keep browsing	Node turns red; every action still works; restart restores it

-

- 23MCCE22 (K.AMULYA